

Week 3 - Day 12b: Sorting

Quick Sort

Check out this interesting diagram which shows how quicksort divides up an example array:

We are going to try this process ourselves by hand, and then later implement a function which performs quicksort.

When in doubt, try drawing a picture like this with your array.

Quick Sort

Exercise 1: Sort by Hand [[]]

We are going to quicksort by hand, step by step, the following list of numbers [10, 15, 1, 2, 6, 12, 5, 7]. (Use the following pivots 7, 5, 10, 12.)

1. The first step in quicksort is to pick a pivot element, for example 7.
2. Then, put all elements less than 7 on the left of 7.
3. Then, put all elements greater than 7 on the right of 7.

Note: the array doesn't have to be completely sorted, but 7 must end up with only items smaller to the left of it.

1.1 Using 7 as a pivot, identify the elements smaller than the pivot and larger than the pivot.

Let's pivot again!

Next, we are going to consider each half of our new array, and run merge sort on each half. We are going to divide our arrays again by picking a new pivot for each half.

For the smaller side, we can use the number 5.

For the greater side, we will use the number 10.

Smaller side

1.2 First, write the half of the array that you found earlier, with items less than 7.

Now, divide this array based on the new pivot 5

Larger side

1.3 First, write the half of the array you found earlier, with the items greater than 7.

Now, divide this array based on the new pivot 10

Continuing the process...

1.4 What happens next, as this process continues? Try to execute the rest of quicksort by hand. When in doubt, try drawing on paper.

Exercise 2: Implement Quicksort [] []

Let's implement quicksort in multiple parts. We need code that takes a list and a pivot, and makes a new list with the smaller elements and another list with the bigger elements.

2.1 Write a function `collect_smaller(lst, p)` that takes a list `lst` and a number `p` and returns those numbers **strictly smaller** than `p`

```
# TODO: Implement the function collect_smaller(lst, p)
def collect_smaller(lst, p):
    # Your code here
    pass

# Test your function
print(collect_smaller([5,2,4,1,9], 2)) # This should give [1]
print(collect_smaller([5,2,4,1,9], 7)) # This should give [5, 2, 4, 1]
```

2.2 Write a function `collect_bigger_or_equal(lst, p)` that takes a list `lst` and a number `p` and returns those numbers **bigger or equal** than `p`

```
# TODO: Implement the function collect_bigger_or_equal(lst, p)
def collect_bigger_or_equal(lst, p):
    # Your code here
    pass

# Test your function
print(collect_bigger_or_equal([5,2,4,1,9], 2)) # This should give [5, 2, 4, 9]
print(collect_bigger_or_equal([5,2,4,1,9], 7)) # This should give [9]
```

We're now ready to implement quicksort.

2.3 Write a function `quicksort(lst)` using recursion that sorts the list `lst` using quicksort. Proceed as follows:

- Base Case: return `lst` if it has 1 or 0 elements
- Select the **last** element of `lst` as the pivot element
- Call `collect_smaller` and `collect_bigger_or_equal` on the list **without the last element** to get two lists: the elements smaller than `p`, and those bigger or equal to `p`
- Sort these two lists by calling `quicksort` recursively
- Piece the two sorted lists together with `p` in the middle, and return this sorted list

```
# TODO: Implement the completed quicksort(lst) function
def quicksort(lst):
    # Your code here
    pass

# Test your function
print(quicksort([5, 3, 7, 9, 6])) # This should give [3, 5, 6, 7, 9]
print(quicksort([4, 5, 5, 4, 4])) # This should give [4, 4, 4, 5, 5]
print(quicksort([1, 2, 3, 4, 5])) # This should give [1, 2, 3, 4, 5]
```

Exercise 3: Analysis []

Let's analyse our algorithm!

If the list is `[1, 2, 4, 3, 5]`, what is happening in the recursion? (Note: We always select the last element as the pivot.)

`quicksort([1, 2, 3, 4, 5])` computes `quicksort([1, 2, 4, 3]) + [5] + quicksort([])`

Then `quicksort([1, 2, 4, 3])` computes `quicksort([1, 2]) + [3] + quicksort([4])`

Then `quicksort([1, 2])` computes `quicksort([1]) + [2] + quicksort([])`

By piecing together the results, we get the sorted list.

3.1 If the list is `[1, 7, 3, 5, 2, 6, 4]`, what is happening in the recursion?

3.2 If the list is `[1, 2, 3, 4, 5, 6, 7]`, what is happening in the recursion?

3.3 In which case do we need to do more work?

Exercise 4: Quicksort Time Complexity []

4.1 If `lst` is of length `n`, what is the time complexity of `collect_smaller(lst)`?

4.2 If `lst` is of length `n`, what is the time complexity of `collect_bigger_or_equal(lst)`?

4.3 If `lst` is of length `n`, what is the time complexity of `quicksort(lst)` **without the recursive step**?

Worst case

4.4 If the input list `lst` for `quicksort(lst)` is already sorted and is of length `n`, how many times do we recurse?

Hint: Look at the example `lst = [1, 2, 3, 4, 5, 6, 7]` from earlier.

4.5 If `lst` is a **sorted** list of length `n`, what is the overall time complexity of `quicksort(lst)`?

Lucky case (HARD)

4.6 If in each step, the pivot partitions `lst` into two lists of equal length, how many times do we recurse?

Hint: Look at the example `lst = [1, 7, 3, 5, 2, 6, 4]` from earlier.

4.7 Assuming `lst` is a list of length `n`, in this scenario, what is the overall time complexity of `quicksort(lst)`?

Congratulations 